COMP-3150 CHAPTERs 6 to 7: LECTURE NOTES

- Chapter 6: BASIC SQL
- Chapter 7: More SQL: Complex Queries, Triggers, Views, and Schema Modification

*** Note that Midterm 2 is written in the class on Thursday, Mar. 5, 2026 in class in Toldo 202. Come to class as usual to write it. The following are the instructions to follow for Midterm 2 so you do not waste too much time going over them during the test:
**

INSTRUCTIONS (Please Read Carefully)
Examination Period is 1 hours 20 minutes.
Answer all questions. Write your answers in the spaces provided in the question paper. This is closed book and closed notes test.
Total Marks =50. Total number of sections = 2
Please read questions carefully! Misinterpreting a question intentionally or unintentionally results in getting a "ZERO" for that question. Good Luck!!!

CONFIDENTIALITY AGREEMENT & STATEMENT OF HONESTY
I confirm that I will keep the content of this assignment/examination confidential.
I confirm that I have not received any unauthorized assistance in preparing for or doing examination. I confirm knowing that a mark of 0 may be assigned for copied work.
For Online Test/Examination in Comp 3150: (additional rules to be observed):

_____ _____
Student Signature Student Name (please print)

_____ _____
Student I.D. Number Date

_____

NOTE THAT I WILL BE GOING OVER IN THE LAST 10 MINUTES OF TODAY'S CLASS YOUR ANSWERS TO THE MIDTERM 2 PRACTICE TEST 2 POSTED FOR YOU. NO FORMAL SOLUTIONS TO ANY PRACTICE TEST IS POSTED ON THE WEB. HOWEVER, IT WILL BE REVIEWED WITH YOU IN THIS CLASS. PARTS OF THE SOLUTION MAY ALSO BE FOUND AT THE END OF THIS LECTURE NOTES.

***

The following questions on Basic SQL discussed in Chapters 6 and 7 of Comp 3150 text book, Chapters 6 and 7, Comp 3150

posted course slide notes, are in-class questions for students to ponder and answer as I teach.

- The answers to the questions are found also by reviewing the Comp 3150, posted power point slide notes for Chapters 6 and 7 and being in class.

- Students are advised to review Chapters 6 and 7 of course book and Comp 3150 posted slide notes before and after each class.

I will also go over the Slide notes in class with examples and integrate them into the class lectures, which are also posted in the More course material link on brightspace.

1. WHAT IS SQL DDL?

   - Data Definition Language (DDL) consists of SQL commands for:

   i) Creating database and table schemas and constructs such as creating database schema (e.g., COMPANY), table schemas (e.g., EMPLOYEE), types, domains, views, assertions, triggers. (Ch 6, slides 1 to 27).

Note that on our Cs server managed by a DBA, the database schema corresponds to our individual Oracle accounts. Each database schema consists of a set of unique table schemas in the database that you can query.

- Main SQL command for DDL is CREATE STATEMENT.

  Example instruction for creating a whole database schema (not table) called COMPANY associated with user Jsmith is given next.

  CREATE SCHEMA COMPANY 'Jsmith';

- An example for creating a table schema for Employee in the Company database is given below:

```
CREATE TABLE EMPLOYEE
        (Fname VARCHAR2(15) NOT NULL,
         Minit CHAR,
         Lname VARCHAR2(15) NOT NULL,
         Ssn CHAR(9) NOT NULL,
        Bdate DATE,
        Address VARCHAR2(25),
        Sex CHAR,
        Salary NUMBER(10, 2),
        Super_ssn CHAR(9),
        Dno NUMBER NOT
        NULL,
        PRIMARY KEY(Ssn));
```

- Fig. 6.1 on page 81 (slides 14 to 16 of Ch 6) shows an example set of CREATE TABLE instructions for the TABLES in the COMPANY database.

- Note that on our cs server Sql|plus system, the data types in the table schemas are defined as follows:

VARCHAR type in the book is VARCHAR2 in cs server
DECIMAL(10, 2) in the book is NUMBER(10, 2) in cs server
INT in the book is NUMBER in cs server
DATE format is cs server is entered in the format: dd-mon-yy

Eg. 12-aug-55

ii) SQL commands for modifying the database and table schemas (Ch. 7, Slides 31 to 35) such as:

a) DROP SCHEMA COMPANY CASCADE;
-- above is used if dropping the database schema (not table schema)

DROP TABLE DEPENDENT CASCADE CONSTRAINTS;
-- The above is for dropping table Dependent and all references to this table.

b) ALTER TABLE COMPANY.DEPARTMENT ALTER Mgr_ssn SET DEFAULT '333445555';

2. WHAT IS SQL DML?

- SQL Data Manipulation Language (DML) is used for loading, deleting, updating and querying data and includes (Slides 44 to 52 of Ch 6):

i) INSERT INTO EMPLOYEE

VALUES ('Franklin', 'T', 'Wong', '333445555', '12-aug-55', '638 Voss, Houston, TX', 'M', 40000, 888665555, 5);

ii) Delete from employee;

iii) UPDATE PROJECT

SET PLOCATION = 'Bellaire', DNUM = 5

WHERE PNUMBER=10;

iv) SQL DML for querying data in the database (Slides 28 to 43 of Ch 6).

- Main DML instruction is the SELECT command for select-project-join (spj) queries with the basic format on book page 188 as:

SELECT <attribute list>
FROM <table list>

WHERE <condition>;

- More expanded version of SELECT command with aggregation is on slide 26 of Ch. 7 as:

SELECT <attribute list>
FROM <table list>
[WHERE <condition>]
[GROUP BY <grouping attribute(s)>
[HAVING <group condition>]
[ORDER BY <attribute list>];

- More complex versions of SELECT command with nesting etc. are seen in Ch. 7 where the main comparison operators for connecting outer and inner queries are IN and EXISTS.

- See the summary of SQL syntax on page 235; slides 36 and 37 of Ch. 7).

**

SQL Querying in Chapters 6 & 7 can be Summarized as follows:

1. Basic SQL querying involving Select-Project-Join (SPJ) as given in the basic Select statement on Slide 29 of Ch. 6.

   SELECT <attribute list>
   FROM <table list>
   WHERE <condition>;

   Here, the <attribute list> is called the project list (that is the set of attributes or expressions in the result of the query), the <table list> is the list of database tables that can be used to answer this query; while WHERE <condition> represents the Selection and Join conditions in the SPJ query.

2. An extended version of the basic SQL query can be found on Slide 43, which includes ordering of query results either in ascending (Asc) (default order) or descending (Desc) order for any attributes in the project list. The format is:

SELECT <attribute list>
FROM <table list>
[WHERE <condition>]
[ORDER BY <attribute list>];

3. Nested Queries as discussed in Ch. 7, Slides 1 to 19.

   Nested Queries are of the form:

   SELECT <attribute list>
   FROM <table list>
   WHERE <condition>

   for the outer query with a nested query in either the FROM clause or the WHERE clause of the outer query.

   - Comparison operators used with nested query are:

   (i) IN, =ANY, =ALL (also >, <, >=, <=, <> in place of = with ANY and ALL)

   (ii) EXISTS

An example query:

Get a list of project numbers for projects that are worked on by employee with last name 'Smith'. Solve using (i) SPJ and (ii) nested form.

(i) SPJ:

```
SELECT W.PNO
FROM WORKS_ON W, EMPLOYEE E
WHERE W.ESSN=E.SSN AND
            E.LNAME='Smith';
```

(ii) Nested version:

(a) Get the Ssn of employee with Lname='Smith'. (inner query)

(b) Get the Pno from the set of Pno's in Works_on where the Ssn from the (a) set is also in the Essn set for these Works_on Pno's.

```
SELECT PNO
FROM WORKS_ON
WHERE ESSN   IN
      (SELECT SSN
```

FROM EMPLOYEE
WHERE LNAME='Smith');

4. Extension of basic SQL query to also include aggregate functions (COUNT, SUM, MAX, MIN, AVG) for summarizing data. This extension also has options with the Grouping and Having clauses as discussed on slide 26 of Chapter 7. The format is:
   SELECT <attribute list>
   FROM <table list>
   [WHERE <condition>]
   [GROUP BY <grouping attribute(s)>
   [HAVING <group condition>]
   [ORDER BY <attribute list>];

5. Full summary of SQL language syntax is provided on slides 36 and 37 of Chapter 7 (as given on page 235 of course book).

**

ABOUT EXPRESSING JOIN IN QUERIES WHEN POSING QUERIES INVOLVING MORE THAN ONE DATABASE TABLE

1. On Slide 31 of Ch. 6, the query 0 is ‘Retrieve the birth date and address of the employee(s) whose name is ‘John B. Smith’.
The instruction to answer this query just from the Employee table is:

```
SELECT Bdate, Address
FROM  EMPLOYEE
WHERE  Fname ='John' AND Minit='B' AND
Lname = 'Smith';
```

- Here is a query (Query 1) that requires more than one table in the database to answer and which requires a table join.

Query 1: Get the name and address of all employees who work for the 'Research' department.

```
SELECT  Fname, Lname, Address
FROM    EMPLOYEE, DEPARTMENT
WHERE   Dname = 'Research'
AND Dnumber = Dno;
```

- In the solution to Query 1 above, in the WHERE clause, the Dname = ‘Research’ is a selection condition. The condition Dnumber = Dno is a join condition that combines two tuples from two tables when the value of the primary key Department.Dnumber is equal to the value of the corresponding foreign key in the second table Employee.Dno.

2. When writing the SQL query, think in terms of three things:

i. The projection list (the result to be retrieved)

ii. The correct tables from the database to retrieve these results from.

iii. The Selection conditions, consisting of the join conditions that connect table attributes and other filtering conditions.

3. The Cross Product of two Tables to be joined.
A join of tables Employee and Department on a common Primary key/ foreign key combination (Department.Dnumber/Employee.Dno) as required on Query 1 on slide 31, comes as a result of the cross product of the two tables (Employee and Department), followed by a selection on the join attributes where Department.Dnumber= Employee.Dno.

- What is a cross product of two tables?
A cross product of the Employee (with 8 tuples) and Department (with 3 tuples) written as Employee x Department, will get all possible combinations of each tuple of Employee with tuples of Department to yield a table with 8 x 3 = 24 tuples. This cross product table also has degree (that is, number of attributes in it) as 14, which is the number of attributes in Employee (10) + number of attributes in Department (4).

- The SQL query 10A on slide 37 gives you the cross product of the tables Employee and Department.

```
SELECT *
from   Employee, Department;
```

While the SQL query below gives the result of the same query when the join condition is applied to get only those combinations where Employee.Dno = Department.Dnumber.

```
SELECT *
from   Employee, Department
where  Employee.Dno = Department.Dnumber;
```

- Note that while the cross product of Employee and Department will retieve 24 tuples, the equi-join query on Employee and Department will retrieve exactly 8 tuples out of the 24 cross product tuples where there is a equivalent match for the primary key and foreign key attributes of the two tables.
- Next, if we discuss the midterm test 2 practice, we would place parts of the solution next in an updated version of this lecture notes.

-

- ** Parts of Solution to Midterm 2 Practice (handed out Winter 2026 Class)

Section B (40 marks):
This section has 3 questions. Answer all questions in this section.

1. **(10 marks)** Given the following relation schema T, with attribute A as the primary key, answer the questions below.

T Relation

A	B	C	D
a1	b1	c1	d1
a2	b2	c2	d1
a3	b2	c1	d1
a4	b2	c3	d1

(i) What functional dependencies FD can you see exist in this relation above?
(ii) What update, (iii) delete and (iv) insertion anomalies may or not be present in this T relation above. Explain with specific examples using FDs and keys in this database.

Que 1 (2.5 marks for each of i to iv for a total of 10 marks)

(i) FDs (2.5 marks)	FD1: A -> {B, C, D} FD2: B -> D; FD3: C -> D; Since for FD2, every tuple with B value b1, its D value is d1 and every tuple with B value b2, its D value is d1. Also, for FD3, for every tuple with C value of c1, it has a D value of d1. This indicates that the primary key does not fully functionally determine all attributes and thus, this table is not in 3NF and contains anomalies. Detailed marking scheme: Duck -0.5 for each of the missing FDs. Duck -2.5 if none is correct.
(ii) Update anomalies (2.5 marks)	Since B -> D, if there is need to change the B value (e.g. b1 has d1 value for its D attribute and we want to change its B value to b3, we must change all other tuples with B value b1 to b3 and have all their D values remain d1), thus several tuples having the B value b1 in the table will have to be updated or there is a violation of the functional dependency (B -> D) in the database that must exist. This is update anomaly. Detailed marking scheme: Duck -2.5 for poor explanation of either redundant multiple update or violation of FDs. Duck -1 if a decent explanation is provided.
(iii) Delete anomalies (2.5 marks)	In the table above, if we delete the 2 tuples with B values of b2, we no longer know what D values are associated with a B value of b2. This is delete anomaly.

	Detailed marking scheme: Duck -2.5 for poor explanation of losing the relationship between attributes of FDs should the last one be deleted. Duck -1 if a decent explanation is provided.
(iv) Insertion anomalies (2.5 marks)	If a new tuple is inserted whose B and D values are not yet known, we might not be able to insert this tuple until we can assign them a B value with matching D value that maintains the FD (B -> D). This is insert anomaly. Thus, we cannot insert a NULL value for B although it is not a key. Detailed marking scheme: Duck -2.5 for poor explanation of not being able to insert NULL value for B or insert the tuple until a B value is known due to the FD B -> A. Duck -1 if a decent explanation is provided.

2. **(15 marks)** Given the following four relations for a simple hospital management record database application where prescriptions given to patients by doctors are tracked for each treatment. Total number of prescriptions received by each patient is known from the patient record at all times.

Patient(pid, pname, address, numprescr)
Doctor(did, dname, specialization, address)
Prescription(prescrid, prescr_name, qty)
Treats(did, pid, prescrid, Pdate)

Here, the primary key for each relation is underlined. The attributes pid stands for patient id, did is doctor id, prescrid is prescription id, numprescr is the cumulative number of prescriptions the patent has received from doctors (e.g., 50). All other attributes are self explanatory.
A small database state of this database is:

PATIENT

pid	pname	address	numprescr
1111	John Smith	Windsor	100
2222	Mary Pert	Windsor	50

DOCTOR

did	dname	specialization	address
1	Mana Patel	Family Physician	Windsor
2	Pat Goodman	Surgeon	London

PRESCRIPTION

prescrid	Prescr_name	Qty
23567	Inflam FX	2
55555	Tylenol	10

TREATS

did	pid	prescrid	Pdate
1	1111	23567	10-Jan-19
2	2222	55555	20-Aug-19

a. Write the SQL DDL instructions to create all 4 tables of the above database in the Oracle DBMS on our cs server specifying all constraints such as Key, entity and foreign key constraints as usual within each instruction (not separately). (**5 marks**)
b. Using only the data in the above instance, write the SQL instructions to load(insert) all the data above in the 4 tables you created. (**5 marks**)

c. Write the SQL query for the following query posed on this database, also write the results of the query and the tables involved in answering the query. (**5 marks**)

Print all patients (pid, pname) treated by doctor 'Mana Patel'

(**5 marks**)

Que 2a (5 marks): Creating Tables

```
CREATE TABLE PATIENT
(PID   VARCHAR2(10) NOT NULL,
Pname VARCHAR2(15),
Address VARCHAR2(15),
Numprescr NUMBER(8),
PRIMARY KEY(Pid));

CREATE TABLE DOCTOR
 (DID VARCHAR2(10) NOT NULL,
Dname VARCHAR2(15),
Specialization  VARCHAR2(15),
Address  VARCHAR2(15),
PRIMARY KEY(DID));

CREATE TABLE PRESCRIPTION
(PRESCRID VARCHAR2(10) NOT NULL,
Prescr_name  VARCHAR2(15),
Qty NUMBER(4),
PRIMARY KEY(PRESCRID));

CREATE TABLE  TREATS
(DID   VARCHAR2(10) NOT NULL,
PID  VARCHAR2(10) NOT NULL,
PRESCRID VARCHAR2(10) NOT NULL,
PDate DATE,
PRIMARY KEY(Did, Pid, PRESCRID),
FOREIGN KEY (Did) REFERENCES DOCTOR(Did),
FOREIGN KEY (Pid) REFERENCES PATIENT(Pid),
FOREIGN KEY (PRESCRID) REFERENCES PRESCRIPTION(PRESCRID));

COMMIT;
```

Detailed marking scheme: It is -1 for each of the 4 tables not created well or -0.5 for any incorrect constraint or attribute up to two for each table.

Que 2b (5 marks): Inserting Data into the Database Tables

Solution (b)

```
INSERT INTO PATIENT
VALUES ('1111', 'John Smith', 'Windsor', 100);
COMMIT;
```

```
INSERT INTO PATIENT
VALUES ('2222', 'Mary Pert', 'Windsor', 500);
COMMIT;

INSERT INTO DOCTOR
VALUES ('1', 'Mana Patel', 'Family Physician', `Windsor');
COMMIT;

INSERT INTO DOCTOR
VALUES ('2', 'Pat Goodman, 'Surgeon', `London');
COMMIT;

INSERT INTO PRESCRIPTION
VALUES ('23567', 'Inflam FX, 2);
COMMIT;

INSERT INTO PRESCRIPTION
VALUES ('55555', 'Tylenol', 10);
COMMIT;

INSERT INTO TREATS
VALUES (`1', '1111',  '23567', '10-Jan-19');
COMMIT;

INSERT INTO TREATS
VALUES ('2222', `2', '55555', '20-Aug-19');
COMMIT;
```

Detailed marking scheme: It is -1 for each of the 4 tables not having all its data loaded well or -0.5 for any line of data with error or missing up to 2 per table.

Question	Query	Tables needed to answer query and Result
2c (5 marks)	Detailed marking scheme: It is 3marks for query, list of tables needed is 1 mark, and result of query is 1 mark. Also, for query (spj) select conditions 1 (even if only one condition missing), project (1 mark) and join conditions (1 mark). Print all patients (pid, pname) treated by doctor 'Mana Patel'. SQL Query is: Select Treats.Pid, Patient.Pname From Patient, Doctor, Treats Where Patient.pid = Treats.pid And Doctor.did = Treats.did And Doctor.dname = 'Mana Patel';	Tables needed are: TREATS, DOCTOR, PATIENT Result of Query is: Pid Pname 1111 John Smith

3. (15 marks)

Given the following WorkPlace relation schema, with Ssn as the primary key, answer the questions below.

WorkPlace

Ssn	name	locker#	rank	level	wage	hours
1	Bob	48	7	M	20	40
2	Bill	22	7	M	20	30
3	Pat	35	5	E	10	30
4	Tony	35	5	E	10	32
5	Namdi	35	7	M	20	40

a. Is this table in 3NF?, If not in 3NF, normalize it into 3NF showing all the tables in your new decomposed database. Explain using functional dependencies. **(6 marks)**

b. Provide one valid tuple instance that can be inserted into this original database. **(3 marks)**

c. Provide one SQL query for updating a record in the original table. **(3 marks)**

d. Provide one SQL query for (i) deleting all tuples in the original table, then (ii) provide one SQL query for deleting the original table schema from your database. **(3 marks)**

Question	Answers
a. (6 marks) Is this table in 3NF?, normalize into 3NF showing all tables Explain with FDs	NO, it is not n 3NF. Violating FD is: rank -> {level, wage} This is because for every tuple with rank value 5, its level is E and its wage is 10 and every tuple with rank value 7, its level is M and its wage is 20. This indicates that the primary key does not determine all non-key attributes non-transitively, and thus, this table is not in 3NF and may contain anomalies. To decompose into 3NF relations, we need to remove the violating FD2 from the original table by deleting the attributes level and wage from the original relation and we create a new relation with the FD2 to have the following two normalized relations. WorkPlace1(Ssn, name, locker#, rank, hours) WorkPlace2 (rank, level, wage) It can be seen that FD2 with refined FD1 are preserved by this decomposition in this database since the FDs in the decomposed database are: FD1: Ssn -> {name, locker#, rank, hours} FD2: rank -> {level, wage} Detailed marking scheme: It is -1 for not knowing table is not in 3NF. It is -2 marks for not discussing well with FDs. It is -3 marks for not correctly decomposing to normal form based on violating FDs.
b. (3 marks)	Any tuple that is inserted has to conform to all the FDs in the database which are FD1 and FD2. Thus, a valid tuple that can be inserted into the original WorkPlace table is: 6 Mimi 37 5 10 50 Detailed marking scheme: It is -3 mark for any insertion causing an anomaly.
c.	UPDATE WorkPlace

(3 marks)	SET hours = 25 WHERE name = 'Bob'; Detailed marking scheme: It is -1 mark for missing each of the 3 parts of Table-name, SET attrs, and where condition.
d. (3 marks) i. ii	DELETE * FROM WorkPlace; DROP TABLE WorkPlace CASCADE; Detailed marking scheme: It is -1.5 mark for missing each of the Delete or Drop table instructions. Note that Drop table can be with or without cascade and may also be in the form of constraint cascade accepted by our system.

**

Que	Ans
1	a
2	a
3	b
4	a
5	b
6	b
7	e

8	c
9	b
10	a

The above is the answers for Section A as discussed in class.